

طرح چند-کمپنی و چند-مشتري

وضعیت: تصویب شده، شروع نشده — نسخه ۲

تاریخ تصمیم: ۱۴۰۵/۰۷/۱۵ (07-10-2026)

برنج در زمان نوشتن: refactor-items-and-company-wise

نسخه انگلیسی: multi-company-plan.md — منسوخ (نسخه ۱)

این سند جایگزین نسخه ۱ است. نسخه ۱ یک مدل تک سطحی پیشنهاد می کرد — تغییر نام branch به company و حذف سطح برنج. آن مدل رد شد، چون نمی تواند انتقال و دیتای مشترک بین برنجهای یک کمپنی را فراهم کند، که نیازمندی صریح است. از نسخه ۱ فقط تصمیماتش در بخش ۱۰ باقی مانده.

۱. خلاصه تصمیم

سه تصمیم، که بقیه سند از آن ها نتیجه می شود:

۱. دو سطح: کمپنی بالای برنج. کمپنی یعنی یک بیزنس با کتابهای مالی خودش؛ برنج یعنی یک محل از همان بیزنس. ۲. مرز اشتراک: بین برنجهای یک کمپنی همه چیز مشترک است (کتلاگ، مشتریان، چارت اکونت، کتابها) و انتقال جنس یک transfer ساده است. بین کمپنی ها هیچ چیز مشترک نیست و هر انتقال سند و اثر حسابداری می خواهد. ۳. SaaS با دیتابیس به ازای هر مشتری: همه چیز روی زیرساخت ما. یک دیتابیس مرکزی مشتریان و اشتراکها را اداره می کند؛ هر مشتری دیتابیس عملیاتی خودش را دارد.

۲. ساختار نهایی

دیتابیس مرکزی (یکی، روی سرور ما)
tenants, plans, subscriptions, licenses,
coupons, invoices, payments, audit_events

دیتابیس هر مشتری (N تا، روی سرور ما)
کمپنی - یک بیزنس: نوع بیزنس، تنظیمات، کتابهای مالی، چارت اکونت،
| کتلاگ اجناس، مشتریان و فروشندگان، اسعار، سال مالی
+ - برنج - یک محل: جواز، صندوق، سری نمبر سند،
| و یک بُعد روی هر تراکنش
+ - گدام - موجودی

قاعده مرز کمپنی

اگر کتابهای مالی، کتلاگ و مشتریان مشترک اند → یک کمپنی با چند برنج. اگر مشترک نیستند → کمپنی های جدا.

در ویزارد، این یک سؤال می شود: «بیزنس تازه است، یا محل دیگری از بیزنس موجود؟» جواب، ساختار را تعیین می کند — و چون ساختار خودش نیت را ثبت می کند، به کالم کمکی (parent_id) ضرورتی نیست.

جواز

هر محل در افغانستان جواز جداگانه لازم دارد، حتی داخل یک بیزنس. پس `license_number` و تاریخ‌های صدور و انقضا روی برنج می‌نشینند، نه کمپنی. در اسناد چاپی، جواز برنج صادرکننده نمایش داده می‌شود.

گروه کمپنی‌ها

ساخته نمی‌شود. چون هر مشتری دیتابیس خودش را دارد، همه کمپنی‌های داخل یک دیتابیس از قبل متعلق به یک مالک‌اند. اگر روزی رپورت گروهی خواسته شد، یک `group_id` اضافه می‌شود.

۳. مرزهای دیتا

دیتا	سطح	توضیح
نوع بیزنس، تنظیمات، تقویم، لوکال	کمپنی	
چارت اکونت و انواع اکونت	کمپنی	هر کمپنی کتاب‌های خودش
اسعار و نرخ تبادل	کمپنی	
کتلاگ اجناس و variantها	کمپنی	بین برنچ‌ها مشترک
مشتریان و فروشندگان	کمپنی	یک مشتری، یک بیلانس، یک سقف اعتبار
سال مالی و financial periods	کمپنی	
کتگوری، برند، واحد، مقدار	کمپنی	
کانفیگ HR (شفت، رخصتی، اجزای معاش، مالیه معاش)	کمپنی	
جواز و تاریخ تجدید	برنج	
صندوق و بانک	برنج	
سری نمبر سند	برنج	
موجودی، lot, piece	گدام	
هر تراکنش	کمپنی + برنج	<code>company_id</code> و <code>branch_id</code> هر دو اجباری

قاعده سراسری: `company_id` روی هر رکورد عملیاتی اجباری است و `branch_id` روی عملیات برنچی. اگر دامنه نامعلوم بود، درخواست رد می‌شود — هرگز به «همه کمپنی‌ها» تبدیل نمی‌شود. انتخاب «همه کمپنی‌ها» فقط در حالت گزارش‌گیری مجاز است؛ ثبت، ویرایش و چاپ سند همیشه در یک کمپنی مشخص.

۴. لایه SaaS

دیتابیس مرکزی

tenants	نام، subdomain، مشخصات اتصال دیتابیس (رمز رمزنگاری شده)، وضعیت (active / grace / read_only / archived)، plan_code
code	plans، نام، قیمت، دوره صورتحساب، limits، features
tenant_id، plan_code	subscriptions، وضعیت، پایان دوره جاری
tenant_id، valid_from، valid_until، revoked_at	licenses

code coupons, نوع (درصد/مبلغ/روز), مقدار, سقف استفاده, اعتبار
 coupon_redemptions کوپون, مشتری, اشتراک, زمان
 invoices / payments صورتحساب و پرداخت
 audit_events چه کسی روی کدام مشتری چه کرد

tenants هم رجیستری مشتریان است و هم **resolver** اتصال. بلاک کردن یعنی تغییر یک ردیف، و درخواست بعدی رد می‌شود.

در دیتابیس مشتری چیزی از لایسنس ذخیره نمی‌شود

نه جدول **licenses**، نه **installations**، و نه کالم **tenant_id**. دیتابیس خودش مرز مشتری است؛ کالمی که در تمام ردیف‌ها یک مقدار دارد صفر معلومات حمل می‌کند و فقط حس امنیت کاذب می‌سازد. entitlement از مرکز خوانده و در کش نگه داشته می‌شود.

بلاک کردن باید نرم باشد

active -> grace (هشدار، همه چیز کار می‌کند)
 read_only -> (دیدن و اکسپورت بله، ثبت سند نه)
 archived -> (فقط اکسپورت)

هیچ‌وقت حذف، هیچ‌وقت قفل کامل. دکانی که وسط ماه از کتاب‌های خودش بیرون بماند دیگر شما را به کسی معرفی نمی‌کند، و **read_only** از نظر فشار پرداخت تقریباً به همان اندازه مؤثر است.

دو منبع جدا برای «آیا این ماژول روشن است؟»

[enabled = business_profile.modules[x
 AND license.features[x
 -> این مشتری HR را نخریده
 -> دواخانه به تولیدی ضرورت ندارد]

دو سؤال متفاوت با دو جواب متفاوت. اگر از اول جدا نشوند، بعداً گره می‌خورند.

سقف عملیاتی

۱۰۰ دیتابیس برای پستگرس مشکلی نیست، ولی تعداد **connection** سقف واقعی است. PgBouncer از همان اول در پلان زیرساخت باشد.

۵. امنیت و ایزولاسیون

در این مدل، مرز امنیتی از **WHERE** به **resolver** اتصال منتقل می‌شود. یک **scope** فراموش‌شده یک جدول را لو می‌دهد؛ یک باگ در انتخاب **connection** کل دیتای یک مشتری را. پس بیشترین تست و دقت باید آنجا باشد.

سه خطری که همین حالا در کد وجود دارد

هر سه امروز تصادفاً کار می‌کنند چون **driver**ها دیتابیسی‌اند (**CACHE_STORE=database**, **QUEUE_CONNECTION=database**)، و روزی که برای مقیاس SaaS به Redis بروید هر سه هم‌زمان و بی‌صدا می‌شکنند:

۱. **ضمیمه‌ها بدون مرز مشتری**. در **AttachmentService** فایل با **'public\$** (**'file->store(\$directory**, روی دیسک **public** می‌نشیند — که وب‌سرور مستقیم سرو می‌کند، بدون چک دسترسی و بدون مشتری در مسیر. در SaaS یعنی اسناد همه مشتریان در یک پوشه عمومی با URL قابل حدس. **اصلاح**: مسیر **../{tenants}/{tenant}**، دیسک **private**، دانلود از مسیر کنترل‌شده و

مجازشده.

۲. کلیدهای کش بدون پیشوند مشتری. کلیدهایی مثل `branches`، `currencies`، `items`، `gl_accounts`، `main_branch` رشته‌های سراسری بدون هیچ دامنه‌ای هستند. روی Redis مشترک، لیست اسعار مشتری A به مشتری B سرو می‌شود. اصلاح: پیشوند مشتری در `CacheKey` و عبور دادن تمام کلیدهای خام از آن.

۳. `job`ها زمینه مشتری را حمل نمی‌کنند. هیچ‌کدام از ۱۲ `job` موجود `tenant_id` ندارند. با `Redis queue`، `job`های همه مشتریان در یک صف می‌ریزند. اصلاح: هر `job` مشتری را حمل کند و اتصال را دوباره برقرار کند.

لایه دوم: RLS پستگرس

`global scope`های Eloquent تنها خط دفاع نباشند. یک کوئری SQL خام یا یک `scope` فراموش‌شده می‌تواند نشت بدهد. Row Level Security روی جدول‌های پرارزش به‌عنوان لایه دوم فعال شود: نقش اپلیکیشن مالک جدول نباشد، و زمینه در تراکنش تنظیم و در اتصال‌های مشترک پاک شود.

کنترل سمت سرور

هر مسیر API، دانلود، ضمیمه، جست‌وجوی خودکار و گزارش باید عضویت کاربر و تطابق دامنه رکورد را چک کند. تغییر دستی `company_id` در درخواست نباید دسترسی ایجاد کند. مجوز در زمان اجرای `job` دوباره بررسی شود.

۶. حسابداری

هر کمپنی دفتر کل، AR/AP، صندوق، بانک و بستن دوره‌اش مستقل است. یک سند حسابداری نمی‌تواند خطوط دو کمپنی داشته باشد. توازن بدهکار و بستانکار در ارز عملیاتی همان کمپنی، هنگام قطعی‌سازی بررسی می‌شود.

انتقال داخل یک کمپنی (بین برنچ‌ها)

انتقال بین گدام کابل و هرات فروش نیست: خروج، موجودی در راه، و رسید گدام ثبت می‌شود. ارزش طبق روش قیمت‌گذاری همان کمپنی حفظ می‌گردد و هزینه انتقال طبق سیاست تخصیص می‌یابد. این دقیقاً همان قابلیت است که این طرح برای آن ساخته شده.

انتقال بین دو کمپنی

خروج مالکیت از الف و ورود به ب. سفارش، فروش/خرید، تحویل، دریافتی و پرداختی متقابل لازم است. یک `intercompany_transaction_id` مشترک همه اسناد را وصل می‌کند. شکست طرف دوم نباید به‌عنوان موفقیت کامل نمایش داده شود.

ارز

مبلغ پایه، ارز، نرخ تأییدشده، جهت نرخ و معادل ارز عملیاتی ذخیره شود. نرخ سند گذشته با تغییر نرخ امروز عوض نمی‌شود. برای حواله: منبع نرخ، تاریخ، کارمزد، مرجع حواله و وضعیت تسویه ثبت شود — حواله در جریان، پول دریافت‌شده نیست.

۷. باگ‌ها و بدهی‌های موجود که پیش‌نیازند

مورد	چرا پیش‌نیاز است
<code>HasSequentialNumber</code> از <code>MAX+1</code> استفاده می‌کند	دو فروش هم‌زمان همان نمبر را می‌گیرند. با دو صندوق همین امروز می‌شکند
<code>sale_items</code> اسنپ‌شات جنس ندارد — فقط <code>item_id</code>	وقتی کتلاگ بین برنچ‌ها مشترک شد، تغییر نام یک جنس ظاهر فاکتورهای گذشته را عوض می‌کند
<code>Company</code> بدون <code>SoftDeletes</code> ولی <code>route</code> دارد	۵۰۰ می‌دهد
<code>Company::search</code> بدون <code>trait HasSearch</code>	صفحه لیست کمپنی‌ها می‌شکند
<code>tenant_id</code> مرده در <code>Account</code> و <code>Currency</code>	کالم در هیچ migration ساخته نشده
<code>branches.sub_domain</code> بلااستفاده	در SaaS، subdomain مال مشتری است نه برنچ؛ بعداً اشتباه گرفته می‌شود
<code>BranchSpecific</code> وقتی برنچ bind نشده از scope بیرون می‌زند	در job و console، ردیف‌های همه را می‌بیند

۸. فازها

فاز	محتوا	خطر
۰	کامند <code>company:audit</code> (چند برنچ زیر هر کمپنی در دیتای واقعی؟ — این اندازه فاز ۳ را تعیین می‌کند) · رفع هر هفت مورد بخش ۷ · رفع سه خطر امنیتی بخش ۵	کم
۱	دیتابیس مرکزی: <code>tenants + resolver</code> اتصال · کامند <code>migrate</code> روی همه دیتابیس‌ها · قلاب <code>entitlement</code> (حتی اگر فعلاً همیشه «بله») · کش، فایل و job مشتری-آگاه	متوسط
۲	کمپنی به‌عنوان سطح: صفات بیزنس روی <code>branches.company_id</code> · <code>companies +</code> جواز · <code>company_id</code> روی ~۹۶ جدول با <code>SetActiveCompany</code> · <code>composite FK</code> · محدود کردن سویچ برنچ — حفره امنیتی اینجا بسته می‌شود · <code>RLS</code> روی جدول‌های پرارزش	متوسط
۳	ادغام <code>master data</code> به سطح کمپنی: چارت اکونت، کتلاگ، مشتریان، اسعار و واحدهای برنچ‌های یک کمپنی در یکی · اسنپ‌شات روی خط سند · تجزیه <code>provisioning</code>	بالا
۴	انتقال جنس بین برنچ‌های یک کمپنی + اکونت موجودی در راه	کم
۵	<code>company_user pivot</code> · سویچر کمپنی · ویزارد ساخت با پیش‌نمایش نوع بیزنس	کم
۶	<code>Settings resolver: config</code> → business profile → کمپنی → یوزر	کم
۷	<code>modules</code> در <code>business profiles</code> ، AND شده با <code>features</code> لایسنس · <code>Spatie teams</code> روی <code>company_id</code>	متوسط
۸	معاملات بین کمپنی: <code>intercompany_transactions</code> و <code>intercompany_documents</code> ، اکونت‌های <code>due-to/due-from</code>	متوسط
۹	پنل مدیریت مشتریان: پکیج، اشتراک، کوپون، فاکتور	کم

فاز ۳ تنها فازی است که دیتای مالی را جابه‌جا می‌کند و به تمرین روی کپی ضرورت دارد. فاز ۲ هم ۵۳ unique index را بازنویسی می‌کند و باید روی بازگردانی بزرگ‌ترین مشتری تمرین شود.

۹. معیار صحت مهاجرت

قبل و بعد از فاز ۳ باید این‌ها دقیقاً تطابق کنند:

- تعداد اسناد و خطوط

- جمع بدهکار و بستانکار
- مانده هر مشتری و فروشنده
- مقدار و ارزش موجودی
- مانده صندوق، بانک و دارایی‌ها

و این سه باید صفر باشند: رکورد بدون کمپنی، مرجع خارج از دامنه، سند بدون خط معتبر. تفاوت‌ها قبل از بازگشایی حل شوند.

آزمون پذیرش

سناریو	نتیجه لازم
تعویض <code>company_id</code> در API	رد دسترسی؛ هیچ عنوان، مبلغ یا ضمیمه‌ای افشا نشود
انتخاب مشتری یا گدام کمپنی دیگر	رد در سرور و دیتابیس، حتی اگر ID دور زده شود
دو تب با دو کمپنی	هر تب فقط در کمپنی خودش ثبت و چاپ کند
دو فروش هم‌زمان و <code>retry</code>	یک سند و یک اثر حسابداری؛ نمبر تکراری صادر نشود
انتقال بین دو برنج یک کمپنی	عاید نسازد؛ موجودی در راه درست ثبت شود
انتقال بین دو کمپنی	اسناد و اکونت‌های متقابل صحیح داشته باشد
ارز و دوره بسته	نرخ تاریخی ثابت؛ قطعی‌سازی در دوره بسته رد شود
دری، پشتو و چاپ	RTL، تاریخ، اعداد، نام کمپنی و برنج در PDF درست

۱۰. تصمیمات ثبت‌شده

تصمیم	چرا
دو سطح، نه یک سطح	مدل تک‌سطحی نمی‌تواند انتقال و دیتای مشترک بین برنچ‌ها بدهد
<code>tenant_id</code> روی جدول‌ها اضافه نشود	دیتابیس خودش مرز است؛ کالم ثابت صفر معلومات دارد و حس امنیت کاذب می‌سازد
گروه کمپنی‌ها ساخته نشود	همه کمپنی‌های یک دیتابیس از قبل یک مالک دارند
چارت اکونت در سطح کمپنی، نه مشترک بین کمپنی‌ها	هر کمپنی کتاب‌های جداگانه دارد؛ اشتراک یعنی بازنویسی <code>transaction_lines</code>
کتلاگ در سطح کمپنی، مشترک بین برنچ‌ها	نیازمندی صریح
جواز روی برنج	هر محل در افغانستان جواز جدا لازم دارد
<code>business_unit</code> فعلاً نه	یک بُعد تحلیلی است، نه یک سطح <code>tenancy</code>
لایسنس در دیتابیس مشتری ذخیره نشود	SaaS است؛ مرکز همیشه در دسترس است
<code>business_type</code> enum + config بماند	«اضافه کردن صنف نباید معنایش <code>migration</code> باشد»
بدون فیلد مالیۀ فروش	در بازار افغانستان ثبت مالیاتی نیست؛ یک تنظیم با پیش‌فرض صفر کافی است

۱۱. چالش‌های آینده

این‌ها حل نشده‌اند و نباید فراموش شوند:

۱. معاملات بین‌کمپنی ناگزیر است. مالک از صندوق یک بیزنس، قرض بیزنس دیگر را می‌دهد. فاز ۸ شروع کار است، نه پایانش. ۲. رپورت ترکیبی خواسته می‌شود حتی بدون مالیات. مالک می‌پرسد «جمع هر دو بیزنس چقدر شد؟» و چون چارت‌ها جدا هستند، به نقشه‌برداری بین چارت‌ها ضرورت دارد. هیچ ارزان: مطمئن شوید چارت پیش‌فرض هر کمپنی عیناً همان slugها را نگه می‌دارد، تا جمع‌زدن فردا یک `group by slug` باشد. ۳. تجربه کاربری با تعداد کمپنی خراب می‌شود. «الآن در کدام کمپنی هستم؟» و پست کردن در کمپنی غلط بی‌صدا و گران است. هیچ ارزان: scope را از روز اول `whereIn('company_id', $ids)` بنویسید با آرایه یک‌عضوی. ۴. `business_type` به‌عنوان config تا جایی مقیاس می‌گیرد. تولیدی به BOM و هتل به اتاق ضرورت دارد — این‌ها flag نیستند، موجودیت‌اند. گروه `modules` باید از اول این را در نظر بگیرد. ۵. دیتابیس-به-ازای-مشتري یعنی هر تغییر `schema` ضرب در تعداد مشتری. نسخه‌ها واگرا می‌شوند و پشتیبانی تبدیل می‌شود به «این مشتری روی کدام نسخه است؟» ۶. ردیف کمپنی به `god-object` تنظیمات تبدیل می‌شود. هر فیچر یک تنظیم اضافه می‌کند و هر تغییر شکل JSON یک migration نرمال‌سازی می‌خواهد.

فایده‌ای که این مدل ممکن می‌کند

دیتابیس مرکزی می‌تواند چیزهای مشترک را به همه مشتریان پوش کند. ملموس‌ترین نمونه برای این بازار: نرخ روزانه افغانی/دالر/کلدار یک بار مرکزی ثبت شود و به همه مشتریان برسد، با امکان override برای هر مشتری.

۱۲. سؤالات باز

۱. در دیتای واقعی مشتریان، چند برنج زیر هر کمپنی وجود دارد؟ این اندازه ادغام فاز ۳ و کل خطر پروژه را تعیین می‌کند. کامند `company:audit` اولین کاری است که باید نوشته شود. ۲. نمرگذاری سند: سری به‌ازای کمپنی با پیشوند برنج (`-INV-KBL`)، یا سری مستقل به‌ازای هر برنج؟ ۳. آیا `read_only` برای مشتری بلاک‌شده کافی است، یا پالیسی سخت‌تری لازم است؟

۱۳. منابع

- Odoo — Multi-company: الگوی اشتراک دیتا و سویچر چند-کمپنی
- PostgreSQL — Row Level Security: لایه دوم محدودسازی
- IFRS IAS 21: ارز عملیاتی و معاملات ارزی
- IFRS 10: تمایز کنترل و تلفیق از جمع ساده گزارش‌ها
- سند طراحی «پشتیبانی چند کمپنی در ERP» نسخه ۱/۰ — مرجع ساختار و آزمون پذیرش